파이썬 객관식 200문항

변수와 수식

제2장 핵심 개념 기반 20문항

문항 1. 다음 중 변수에 대한 설명으로 가장 옳은 것은?

① 변수는 한 번 저장한 값을 바꿀 수 없다.
② 변수는 값을 저장해 두는 이름이 있는 공간이다.
③ 변수는 숫자만 저장할 수 있다.
④ 변수는 함수 안에서만 사용할 수 있다.

정답	②
모범답안	② 변수는 데이터를 저장하고 다시 사용하기 위한 이름이 있는 저장공간이다.
해설	파이썬의 변수는 숫자, 문자열 등 다양한 값을 저장할 수 있으며, 같은 변수에 새로운 값을 다시 대입할 수도 있다.

문항 2. 다음 중 올바른 변수 이름은?

① 2score
② my-score
③ class
④ my_score

정답	④
모범답안	④ my_score는 식별자 규칙에 맞는 변수 이름이다.
해설	변수 이름은 숫자로 시작할 수 없고, 하이픈(-)을 사용할 수 없으며, 예약어도 사용할 수 없다.

문항 3. 다음 코드의 실행 결과는 무엇인가?

```
x = 5
y = 2
print(x + y * 3)
```

① 11

② 21

③ 15

④ 9

정답	①
모범답안	① 11이 출력된다.
해설	곱셈이 덧셈보다 먼저 계산되므로 y * 3 = 6, 그 다음 x + 6 = 11이 된다.

문항 4. 다음 중 대입문의 의미를 가장 잘 설명한 것은?

① 왼쪽 값과 오른쪽 값이 항상 같다는 뜻이다.

② 오른쪽 값을 왼쪽 변수에 저장한다는 뜻이다.

③ 왼쪽 값을 오른쪽 식에 더한다는 뜻이다.

④ 수학식과 완전히 같은 의미이다.

정답	②
모범답안	② 오른쪽의 값을 계산한 뒤 왼쪽 변수에 저장한다.
해설	파이썬의 = 는 수학의 등호가 아니라 대입 연산자이다. 따라서 x = x + 1 같은 문장도 가능하다.

문항 5. 다음 코드 실행 후 x의 값은?

```
x = 10
x += 4
```

① 6

② 10

③ 14

④ 40

정답	③
모범답안	③ x의 값은 14이다.
해설	복합 대입 연산자 x += 4는 x = x + 4와 같은 뜻이다.

문항 6. 다음 중 10을 3으로 나누었을 때의 몫을 구하는 연산식은?

① 10 / 3

② 10 // 3

③ 10 % 3

④ 10 ** 3

정답	②
모범답안	② 10 // 3은 정수 몫을 구한다.
해설	/ 는 실수 나눗셈, // 는 몫, % 는 나머지, ** 는 거듭제곱이다.

문항 7. 다음 코드의 실행 결과는?

```
print(10 % 3)
```

① 0
② 1
③ 3
④ 3.3333

정답	②
모범답안	② 나머지 1이 출력된다.
해설	% 연산자는 나눗셈의 나머지를 구한다. 10을 3으로 나누면 몫은 3, 나머지는 1이다.

문항 8. 다음 중 자료형 변환 함수로 옳은 것은?

① change()
② number()
③ int()
④ value()

정답	③
모범답안	③ int()는 문자열 등을 정수로 변환할 때 사용한다.
해설	파이썬에서는 int(), float(), str() 같은 내장 함수를 이용해 자료형을 바꾼다.

문항 9. 다음 코드의 실행 결과는?

```
x = '10'
y = 5
print(x + str(y))
```

① 15
② 105
③ 10 5

④ 오류 발생

정답	②
모범답안	② 문자열 '10'과 '5'가 이어져 105가 출력된다.
해설	문자열과 숫자는 바로 더할 수 없으므로 y를 str(y)로 바꿔 문자열 연결을 수행했다.

문항 10. 다음 코드의 실행 결과는?

```
print(2 ** 3)
```

① 5

② 6

③ 8

④ 9

정답	③
모범답안	③ 2의 3제곱인 8이 출력된다.
해설	** 연산자는 거듭제곱을 뜻한다.

문항 11. 다음 중 입력받은 값을 정수로 바꾸는 코드로 가장 알맞은 것은?

① x = print(input())

② x = int(input())

③ x = input(print())

④ x = str(input())

정답	②
모범답안	② int(input()) 형태가 적절하다.
해설	input()은 문자열을 돌려주므로 숫자 계산을 하려면 int()로 감싸야 한다.

문항 12. 다음 코드의 실행 결과는?

```
x = 3
y = 4
print((x + y) * 2)
```

① 10

② 11

③ 14

④ 24

정답	③
모범답안	③ 괄호 안을 먼저 계산하므로 14가 출력된다.
해설	연산자 우선순위에서 괄호가 가장 먼저 처리된다. 3 + 4 = 7, 그 다음 7 * 2 = 14이다.

문항 13. 다음 중 예약어이므로 변수 이름으로 사용할 수 없는 것은?

① total

② value1

③ for

④ user_name

정답	③
모범답안	③ for는 파이썬 예약어이다.
해설	예약어는 이미 언어에서 특별한 의미로 사용되므로 변수 이름으로 쓸 수 없다.

문항 14. 다음 코드의 실행 결과는?

```
price = 1200
count = 3
print(price * count)
```

① 1200

② 3

③ 3600

④ 123

정답	③
모범답안	③ 총액 3600이 출력된다.
해설	변수에 저장된 값끼리도 일반 수식처럼 곱셈할 수 있다.

문항 15. 다음 코드 실행 후 결과로 가장 알맞은 것은?

```
x = 7
x = x - 2
print(x)
```

① 5
② 7
③ 9
④ 오류 발생

정답	①
모범답안	① x는 5가 된다.
해설	오른쪽 식 x - 2를 먼저 계산한 뒤 그 결과를 다시 x에 저장한다.

문항 16. 다음 중 실수(float) 결과가 나오는 식은?

① 8 // 2
② 8 % 2
③ 8 / 2
④ 8 - 2

정답	③
모범답안	③ / 연산은 실수 나눗셈을 수행한다.
해설	파이썬에서 / 는 결과가 정수로 딱 나누어떨어져도 실수형으로 계산한다.

문항 17. 다음 코드의 실행 결과는?

```
name = 'Py'
print(name * 3)
```

① PyPyPy
② Py3
③ Py Py Py
④ 오류 발생

정답	①
모범답안	① 문자열 반복 결과인 PyPyPy가 출력된다.
해설	문자열 * 정수는 해당 문자열을 정수 횟수만큼 반복한다.

문항 18. 다음 중 연산자 우선순위에 대한 설명으로 옳은 것은?

① 덧셈이 곱셈보다 먼저 계산된다.
② 곱셈과 나눗셈이 덧셈과 뺄셈보다 먼저 계산된다.
③ 모든 연산자는 항상 왼쪽에서 오른쪽으로만 계산된다.

④ 괄호는 우선순위에 영향을 주지 않는다.

정답	②
모범답안	② 곱셈과 나눗셈이 덧셈과 뺄셈보다 우선한다.
해설	기본 우선순위를 바꾸고 싶을 때는 괄호를 사용한다.

문항 19. 다음 코드의 실행 결과는?

```
x = 5
y = 2
print(float(x) / y)
```

① 2
② 2.0
③ 2.5
④ 3.0

정답	③
모범답안	③ 2.5가 출력된다.
해설	x를 float로 바꿔도 핵심은 5를 2로 나눈 실수 결과가 2.5라는 점이다.

문항 20. 다음 중 변수와 수식 단원에서 가장 적절한 설명은?

① 프로그램의 모든 문장은 조건문으로만 이루어진다.
② 변수는 데이터를 저장하고 수식은 그 데이터를 계산하는 데 사용된다.
③ 수식은 문자열에서만 사용할 수 있다.
④ 변수는 반드시 입력문과 함께 써야 한다.

정답	②
모범답안	② 변수는 값 저장, 수식은 계산이라는 역할을 가진다.
해설	변수와 수식은 이후 조건문, 반복문, 함수 학습의 기초가 되는 핵심 요소이다.

조건문

제3장 핵심 개념 기반 50문항

문항 1. 다음 중 if문의 올바른 형식은?

① if x > 0

② if x > 0:

③ if (x > 0) then

④ if: x > 0

정답	②
모범답안	② 조건식 뒤에는 반드시 콜론(:)이 온다.
해설	파이썬의 if문은 if 조건식: 형태로 작성하며, 실행문은 들여쓴다.

문항 2. 다음 코드의 출력 결과는?

```
x = 7
if x > 5:
    print('A')
else:
    print('B')
```

① A

② B

③ A와 B

④ 출력 없음

정답	①
모범답안	① A가 출력된다.
해설	7 > 5는 참이므로 if 블록이 실행된다.

문항 3. 다음 코드의 출력 결과는?

```
x = 3
if x > 5:
    print('A')
```

```
else:
    print('B')
```

① A

② B

③ A와 B

④ 오류 발생

정답	②
모범답안	② B가 출력된다.
해설	3 > 5는 거짓이므로 else 블록이 실행된다.

문항 4. 다음 중 관계연산자가 아닌 것은?

① <

② >=

③ ==

④ +

정답	④
모범답안	④ + 는 산술연산자이다.
해설	관계연산자는 값의 크기나 같고 다름을 비교한다.

문항 5. 다음 중 5와 5가 같은지 비교하는 식은?

① 5 = 5

② 5 == 5

③ 5 != 5

④ 5 >= 6

정답	②
모범답안	② == 연산자는 값의 같음을 비교한다.
해설	= 는 대입 연산자, == 는 비교 연산자이다.

문항 6. 논리연산자 and에 대한 설명으로 옳은 것은?

① 둘 중 하나만 참이면 참

② 둘 다 참이어야 참

③ 항상 참을 반환

④ 항상 거짓을 반환

정답	②
모범답안	② and는 두 조건이 모두 참일 때만 참이다.

해설	복합 조건을 만들 때 자주 사용한다.

문항 7. 논리연산자 or에 대한 설명으로 옳은 것은?

① 둘 중 하나라도 참이면 참
② 둘 다 참이어야 참
③ 항상 거짓이다
④ not와 같은 뜻이다

정답	①
모범답안	① or는 두 조건 중 하나라도 참이면 참이다.
해설	폭넓은 조건을 허용할 때 사용한다.

문항 8. not True의 결과는 무엇인가?

① True
② False
③ 1
④ 오류

정답	②
모범답안	② not True는 False이다.
해설	not은 참과 거짓을 뒤집는다.

문항 9. 다음 코드의 출력 결과는?

```
x = 10
y = 20
if x < 15 and y > 15:
    print('YES')
else:
    print('NO')
```

① YES
② NO
③ 둘 다 출력
④ 출력 없음

정답	①
모범답안	① YES가 출력된다.
해설	두 조건이 모두 참이므로 and 전체 결과도 참이다.

문항 10. 다음 코드의 출력 결과는?

```
x = 2
if x > 5 or x < 3:
    print('YES')
else:
    print('NO')
```

① YES
② NO
③ 둘 다 출력
④ 오류

정답	①
모범답안	① YES가 출력된다.
해설	x > 5는 거짓이지만 x < 3은 참이므로 or 전체가 참이다.

문항 11. 다음 중 elif를 사용하는 이유로 가장 적절한 것은?

① 반복을 만들기 위해
② 여러 조건을 순서대로 검사하기 위해
③ 변수를 선언하기 위해
④ 함수를 호출하기 위해

정답	②
모범답안	② elif는 다중 선택 구조를 만들 때 사용한다.
해설	if-elif-else 구조를 이용하면 조건을 단계적으로 검사할 수 있다.

문항 12. 다음 코드의 출력 결과는?

```
score = 85
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
```

```
else:
    print('C')
```

① A
② B
③ C
④ 오류

정답	②
모범답안	② B가 출력된다.
해설	85는 90 이상은 아니지만 80 이상이므로 elif 블록이 실행된다.

문항 13. 다음 코드의 출력 결과는?

```
score = 72
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
else:
    print('C')
```

① A
② B
③ C
④ 출력 없음

정답	③
모범답안	③ C가 출력된다.
해설	앞선 조건이 모두 거짓이므로 else가 실행된다.

문항 14. 다음 중 조건문의 블록을 구분하는 기준은?

① 세미콜론
② 중괄호
③ 들여쓰기
④ 괄호

정답	③
모범답안	③ 파이썬은 들여쓰기로 블록을 구분한다.

해설	들여쓰기가 맞지 않으면 문법 오류가 발생할 수 있다.

문항 15. 다음 코드의 출력 결과는?

```
x = 8
if x % 2 == 0:
    print('짝수')
else:
    print('홀수')
```

① 짝수
② 홀수
③ 0
④ 오류

정답	①
모범답안	① 짝수가 출력된다.
해설	8을 2로 나눈 나머지가 0이므로 조건이 참이다.

문항 16. 다음 중 중첩 if문의 설명으로 옳은 것은?

① if문 안에는 if문을 사용할 수 없다.
② if문 안에 또 다른 if문을 넣을 수 있다.
③ 반복문 안에서만 가능하다.
④ elif와 함께 사용할 수 없다.

정답	②
모범답안	② 중첩 if는 if문 안에 또 다른 if문이 있는 구조이다.
해설	세부 조건을 한 번 더 검사할 때 사용한다.

문항 17. 다음 코드의 출력 결과는?

```
x = 10
if x > 0:
    if x < 20:
        print('OK')
```

① OK

② 10

③ True

④ 출력 없음

정답	①
모범답안	① OK가 출력된다.
해설	바깥 조건과 안쪽 조건이 모두 참이다.

문항 18. 다음 코드의 출력 결과는?

```
x = -1
if x > 0:
    print('양수')
elif x == 0:
    print('0')
else:
    print('음수')
```

① 양수

② 0

③ 음수

④ 오류

정답	③
모범답안	③ 음수가 출력된다.
해설	x가 -1이므로 앞선 조건이 거짓이고 else가 실행된다.

문항 19. 다음 중 삼항 조건 연산자의 올바른 형식은?

① a ? b : c

② b if a else c

③ if a then b else c

④ a if b : c

정답	②
모범답안	② 파이썬의 삼항 연산자는 b if a else c 형태이다.
해설	조건이 참이면 b, 거짓이면 c가 선택된다.

문항 20. 다음 코드의 출력 결과는?

```
x = 4
msg = '크다' if x > 5 else '작다'
print(msg)
```

① 크다
② 작다
③ 4
④ 오류

정답	②
모범답안	② '작다'가 출력된다.
해설	4 > 5는 거짓이므로 else 쪽 값이 선택된다.

문항 21. 다음 중 조건식으로 가장 적절하지 않은 것은?

① x > 3
② a == b
③ score >= 60 and age >= 20
④ print('hello')

정답	④
모범답안	④ print('hello')는 출력 함수 호출이지 비교 결과를 만드는 식이 아니다.
해설	조건문에는 보통 True/False로 판단 가능한 식을 넣는다.

문항 22. 다음 코드의 출력 결과는?

```
x = 5
if not x > 3:
    print('YES')
else:
    print('NO')
```

① YES
② NO
③ 둘 다 출력
④ 오류

정답	②
모범답안	② NO가 출력된다.
해설	x > 3은 참이고, not을 붙이면 거짓이 된다.

문항 23. 다음 코드의 출력 결과는?

```
temp = 30
if temp >= 30:
    print('덥다')
elif temp >= 20:
    print('선선하다')
else:
    print('춥다')
```

① 덥다
② 선선하다
③ 춥다
④ 오류

정답	①
모범답안	① 덥다가 출력된다.
해설	첫 번째 조건이 참이면 아래 elif는 검사하지 않는다.

문항 24. 다음 중 else에 대한 설명으로 옳은 것은?

① 조건식을 반드시 가진다.
② 앞의 모든 조건이 거짓일 때 실행된다.
③ 반복문에서만 사용된다.
④ if 없이 단독으로 사용한다.

정답	②
모범답안	② else는 앞의 if/elif 조건이 모두 거짓일 때 실행된다.
해설	else 자체는 별도의 조건식을 쓰지 않는다.

문항 25. 다음 코드의 출력 결과는?

```
money = 5000
```

```
if money >= 10000:
    print('택시')
else:
    print('버스')
```

① 택시
② 버스
③ 둘 다 출력
④ 오류

정답	②
모범답안	② 버스가 출력된다.
해설	5000은 10000 이상이 아니므로 else가 실행된다.

문항 26. 다음 코드의 출력 결과는?

```
x = 9
if x % 3 == 0 and x % 2 == 0:
    print('A')
else:
    print('B')
```

① A
② B
③ A와 B
④ 출력 없음

정답	②
모범답안	② B가 출력된다.
해설	9는 3의 배수이지만 2의 배수는 아니므로 and 결과가 거짓이다.

문항 27. 다음 코드의 출력 결과는?

```
a = 10
b = 20
if a > b:
    print('a')
```

```
elif a < b:
    print('b')
else:
    print('same')
```

① a
② b
③ same
④ 오류

정답	②
모범답안	② b가 출력된다.
해설	10 < 20이 참이므로 elif 블록이 실행된다.

문항 28. 다음 중 올바른 설명은?

① elif는 if 없이 단독 사용 가능하다.
② else는 조건식을 가진다.
③ else는 앞의 조건이 거짓일 때 실행된다.
④ if문에서는 비교연산자를 사용할 수 없다.

정답	③
모범답안	③ else는 이전 조건이 성립하지 않을 때 실행된다.
해설	elif와 else는 반드시 앞에 if가 있어야 연결된다.

문항 29. 다음 코드의 출력 결과는?

```
age = 20
if age >= 20:
    print('성인')
else:
    print('미성년')
```

① 성인
② 미성년
③ 20
④ 오류

정답	①
모범답안	① 성인이 출력된다.
해설	20 이상이므로 if 조건이 참이다.

문항 30. 다음 중 짝수 판별 조건식으로 옳은 것은?

① x / 2 == 0

② x % 2 == 0

③ x // 2 == 0

④ x - 2 == 0

정답	②
모범답안	② x % 2 == 0이 짝수 판별식이다.
해설	2로 나눈 나머지가 0이면 짝수이다.

문항 31. 다음 코드의 출력 결과는?

```
x = 0
if x:
    print('참')
else:
    print('거짓')
```

① 참

② 거짓

③ 0

④ 오류

정답	②
모범답안	② 거짓이 출력된다.
해설	숫자 0은 조건식에서 False로 취급된다.

문항 32. 다음 코드의 출력 결과는?

```
name = ''
if name:
    print('입력됨')
else:
    print('비어 있음')
```

① 입력됨

② 비어 있음

③ 둘 다

④ 오류

정답	②
모범답안	② 비어 있음이 출력된다.
해설	빈 문자열은 False로 평가된다.

문항 33. 다음 코드의 출력 결과는?

```
x = 6
if x > 3:
    print('A')
if x > 5:
    print('B')
```

① A만 출력

② B만 출력

③ A와 B 모두 출력

④ 출력 없음

정답	③
모범답안	③ A와 B가 모두 출력된다.
해설	이 코드는 if-elif가 아니라 독립된 if 두 개이므로 둘 다 검사한다.

문항 34. 다음 코드의 출력 결과는?

```
score = 95
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
elif score >= 70:
    print('C')
else:
    print('D')
```

① A

② B

③ C

④ D

정답	①
모범답안	① A가 출력된다.
해설	가장 먼저 참이 된 분기 하나만 실행된다.

문항 35. 다음 코드의 출력 결과는?

```
x = 4
y = 7
if x > 5 or y > 5:
    print('참')
else:
    print('거짓')
```

① 참

② 거짓

③ 둘 다

④ 오류

정답	①
모범답안	① 참이 출력된다.
해설	x > 5는 거짓이지만 y > 5가 참이므로 전체는 참이다.

문항 36. 다음 코드의 출력 결과는?

```
x = 4
y = 7
if x > 5 and y > 5:
    print('참')
else:
    print('거짓')
```

① 참

② 거짓

③ 둘 다

④ 출력 없음

정답	②
모범답안	② 거짓이 출력된다.
해설	and는 두 조건이 모두 참이어야 하므로 하나라도 거짓이면 전체는 거짓이다.

문항 37. 다음 중 비교 결과가 True인 것은?

① 3 > 5

② 7 <= 7

③ 4 != 4

④ 2 == 3

정답	②
모범답안	② 7 <= 7은 참이다.
해설	<= 는 작거나 같다를 의미하므로 같은 값도 참이다.

문항 38. 다음 코드의 출력 결과는?

```
x = 12
if x % 3 == 0:
    if x % 4 == 0:
        print('둘 다')
    else:
        print('3의 배수')
```

① 둘 다

② 3의 배수

③ 출력 없음

④ 오류

정답	①
모범답안	① 둘 다가 출력된다.
해설	12는 3의 배수이면서 4의 배수이기도 하다.

문항 39. 다음 코드의 출력 결과는?

```
x = 15
```

```
if x < 10:
    print('작다')
elif x < 20:
    print('중간')
else:
    print('크다')
```

① 작다
② 중간
③ 크다
④ 오류

정답	②
모범답안	② 중간이 출력된다.
해설	15는 10보다 작지 않지만 20보다 작으므로 elif가 참이다.

문항 40. 다음 중 들여쓰기 오류와 가장 관련이 깊은 것은?

① 변수명
② 함수명
③ 조건문의 실행 범위
④ 산술연산 결과

정답	③
모범답안	③ 들여쓰기는 조건문에서 어떤 문장이 블록에 속하는지 결정한다.
해설	파이썬은 들여쓰기를 문법 요소로 사용한다.

문항 41. 다음 코드의 출력 결과는?

```
x = 2
if x == 1:
    print('one')
elif x == 2:
    print('two')
else:
    print('other')
```

① one

② two

③ other

④ 오류

정답	②
모범답안	② two가 출력된다.
해설	x 값이 2와 같으므로 두 번째 분기가 실행된다.

문항 42. 다음 중 조건문 작성 시 가장 적절한 설명은?

① 조건식 뒤에 세미콜론을 쓴다.

② 조건식 뒤에 콜론을 쓰고 본문을 들여쓴다.

③ 조건식은 반드시 괄호만 사용한다.

④ else 뒤에는 다시 조건식을 쓴다.

정답	②
모범답안	② 파이썬 조건문 문법의 핵심이다.
해설	콜론과 들여쓰기는 파이썬 제어문의 기본 형식이다.

문항 43. 다음 코드의 출력 결과는?

```
x = 100
if x >= 100:
    print('pass')
```

① pass

② 출력 없음

③ 100

④ 오류

정답	①
모범답안	① pass가 출력된다.
해설	조건이 참이므로 if 블록이 실행된다.

문항 44. 다음 코드의 출력 결과는?

```
x = 1
y = 0
if x and y:
```

```
    print('둘 다 참')
else:
    print('거짓 포함')
```

① 둘 다 참
② 거짓 포함
③ 1
④ 0

정답	②
모범답안	② 거짓 포함이 출력된다.
해설	0은 False이므로 and 전체 결과는 거짓이다.

문항 45. 다음 코드의 출력 결과는?

```
x = 1
y = 0
if x or y:
    print('참 있음')
else:
    print('둘 다 거짓')
```

① 참 있음
② 둘 다 거짓
③ 1
④ 0

정답	①
모범답안	① 참 있음이 출력된다.
해설	x가 1이므로 True로 간주되어 or 전체 결과가 참이다.

문항 46. 다음 코드의 출력 결과는?

```
num = 11
if num % 2 == 0:
    print('짝수')
```

```
else:
    print('홀수')
```

① 짝수

② 홀수

③ 11

④ 오류

정답	②
모범답안	② 홀수가 출력된다.
해설	11을 2로 나눈 나머지는 1이므로 홀수이다.

문항 47. 다음 코드의 출력 결과는?

```
x = 5
if x > 0:
    print('양수')
else:
    print('0 또는 음수')
```

① 양수

② 0 또는 음수

③ 둘 다

④ 오류

정답	①
모범답안	① 양수가 출력된다.
해설	5는 0보다 크므로 조건이 참이다.

문항 48. 다음 코드의 출력 결과는?

```
x = -5
if x > 0:
    print('양수')
else:
    print('0 또는 음수')
```

① 양수

② 0 또는 음수

③ 둘 다

④ 오류

정답	②
모범답안	② 0 또는 음수가 출력된다.
해설	-5는 0보다 크지 않다.

문항 49. 다음 중 여러 개의 독립적인 조건을 각각 검사하려면 가장 적절한 것은?

① if 하나만 사용

② if-elif-else 하나만 사용

③ 서로 독립된 if문 여러 개 사용

④ else만 사용

정답	③
모범답안	③ 독립적이면 if문을 각각 따로 써야 한다.
해설	if-elif-else는 하나의 분기만 실행되는 상호배타적 구조에 적합하다.

문항 50. 다음 코드의 출력 결과는?

```
x = 12
if x > 10:
    print('A')
else:
    print('B')
print('C')
```

① A

② A와 C

③ B와 C

④ C만

정답	②
모범답안	② A와 C가 출력된다.
해설	if문이 끝난 뒤 아래의 print('C')는 항상 실행된다.

반복문

제4장 핵심 개념 기반 50문항

문항 1. 다음 중 반복문의 주된 목적은 무엇인가?

① 코드를 무조건 길게 만들기 위해
② 같은 작업을 여러 번 수행하기 위해
③ 변수 선언을 없애기 위해
④ 오류를 자동으로 수정하기 위해

정답	②
모범답안	② 같은 작업을 반복 실행하기 위해 사용한다.
해설	반복문을 이용하면 같은 코드를 여러 줄 쓰지 않아도 된다.

문항 2. 다음 중 횟수가 정해진 반복에 가장 적절한 문장은?

① if
② for
③ while
④ def

정답	②
모범답안	② for문이 적절하다.
해설	for문은 range()나 문자열, 리스트 등을 순회할 때 자주 사용된다.

문항 3. 다음 코드의 출력 결과는?

```
for i in range(3):
    print(i)
```

① 0 1 2
② 1 2 3
③ 0 1 2 3
④ 3

정답	①
모범답안	① 0, 1, 2가 차례로 출력된다.
해설	range(3)은 0부터 2까지 생성한다.

문항 4. range(5)가 생성하는 값은?

① 1,2,3,4,5

② 0,1,2,3,4

③ 0,1,2,3,4,5

④ 5

정답	②
모범답안	② 0부터 4까지 생성한다.
해설	range의 끝 값은 포함되지 않는다.

문항 5. 다음 코드의 출력 결과는?

```
for i in range(1,4):
    print(i)
```

① 1 2 3

② 0 1 2 3

③ 1 2 3 4

④ 4

정답	①
모범답안	① 1, 2, 3이 출력된다.
해설	range(1,4)는 1 이상 4 미만의 정수를 만든다.

문항 6. 다음 중 range(2,10,2)의 결과로 옳은 것은?

① 2,4,6,8

② 2,4,6,8,10

③ 0,2,4,6,8

④ 1,3,5,7,9

정답	①
모범답안	① 2,4,6,8이 생성된다.
해설	간격이 2이므로 2씩 증가하며 끝 값 10은 포함하지 않는다.

문항 7. 다음 코드의 실행 결과는?

```
s = 0
for i in range(1,4):
```

```
    s += i
print(s)
```

① 3

② 6

③ 7

④ 10

정답	②
모범답안	② 합계 6이 출력된다.
해설	1 + 2 + 3 = 6이다.

문항 8. while문에 대한 설명으로 옳은 것은?

① 항상 10번 반복한다.

② 조건이 참인 동안 반복한다.

③ 문자열에는 사용할 수 없다.

④ for문 안에서만 쓸 수 있다.

정답	②
모범답안	② while문은 조건 반복 구조이다.
해설	조건이 거짓이 될 때까지 계속 실행된다.

문항 9. 다음 코드의 출력 결과는?

```
i = 0
while i < 3:
    print(i)
    i += 1
```

① 0 1 2

② 1 2 3

③ 0 1 2 3

④ 무한 반복

정답	①
모범답안	① 0,1,2가 출력된다.
해설	i를 1씩 증가시키므로 3이 되면 반복이 끝난다.

문항 10. 다음 코드가 무한 루프가 되는 이유는?

```
i = 0
while i < 3:
    print(i)
```

① print가 있기 때문에
② i가 정수이기 때문에
③ i 값이 바뀌지 않기 때문에
④ while문은 원래 무한 반복이기 때문에

정답	③
모범답안	③ i가 증가하지 않아 조건이 계속 참이 된다.
해설	조건을 바꾸는 코드가 없으면 while문은 끝나지 않을 수 있다.

문항 11. 다음 중 무한 루프를 만드는 대표적인 형태는?

① for i in range(3):
② while True:
③ if True:
④ elif x > 0:

정답	②
모범답안	② while True:는 별도 종료가 없으면 계속 반복된다.
해설	반복문 안에서 break 등으로 빠져나올 수 있다.

문항 12. 다음 코드의 출력 결과는?

```
for ch in 'ABC':
    print(ch)
```

① A B C
② ABC
③ 0 1 2
④ 오류

정답	①
모범답안	① 문자열의 각 문자가 한 줄씩 출력된다.
해설	for문은 문자열도 순회할 수 있다.

문항 13. 반복문을 즉시 종료시키는 명령은?

① stop

② exit

③ break

④ continue

정답	③
모범답안	③ break는 현재 반복문 전체를 종료한다.
해설	가장 가까운 반복문 하나를 빠져나간다.

문항 14. 다음 코드의 출력 결과는?

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

① 0 1 2

② 0 1 2 3

③ 1 2 3

④ 0 1 2 3 4

정답	①
모범답안	① 0,1,2가 출력된다.
해설	i가 3이 되는 순간 break로 반복이 종료된다.

문항 15. 현재 반복만 건너뛰고 다음 반복으로 넘어가는 명령은?

① pass

② break

③ continue

④ return

정답	③
모범답안	③ continue는 현재 반복의 남은 부분만 건너뛴다.
해설	반복문 전체를 끝내는 것은 break이다.

문항 16. 다음 코드의 출력 결과는?

```
for i in range(5):
    if i == 2:
```

```
        continue
    print(i)
```

① 0 1 2 3 4

② 0 1 3 4

③ 2 3 4

④ 0 1 2

정답	②
모범답안	② 0,1,3,4가 출력된다.
해설	i가 2일 때만 print를 건너뛴다.

문항 17. 다음 코드의 출력 결과는?

```
for i in range(2):
    for j in range(2):
        print(i, j)
```

① (0,0)만 출력

② 0 0 / 0 1 / 1 0 / 1 1

③ 0 1 / 1 2

④ 오류

정답	②
모범답안	② 네 가지 조합이 모두 출력된다.
해설	중첩 반복문에서 안쪽 반복문은 바깥 반복문 1회마다 모두 실행된다.

문항 18. 중첩 반복문에 대한 설명으로 옳은 것은?

① 반복문 안에 다른 반복문을 넣을 수 없다.

② 바깥 반복 1번마다 안쪽 반복이 모두 실행된다.

③ while문에서만 가능하다.

④ 반드시 3중 이상이어야 한다.

정답	②
모범답안	② 이것이 중첩 반복문의 기본 동작이다.
해설	행과 열을 다루는 문제에서 특히 자주 쓰인다.

문항 19. 다음 코드의 출력 결과는?

```
for i in range(3):
    print('*')
```

① 별 2개
② 별 3개
③ 별 4개
④ 오류

정답	②
모범답안	② 별이 3번 출력된다.
해설	range(3)이므로 3회 반복이다.

문항 20. 다음 코드의 출력 결과는?

```
for i in range(1,6):
    if i % 2 == 0:
        print(i)
```

① 1 3 5
② 2 4
③ 1 2 3 4 5
④ 0 2 4

정답	②
모범답안	② 2와 4가 출력된다.
해설	1부터 5까지 중 짝수만 골라 출력했다.

문항 21. 다음 코드의 출력 결과는?

```
i = 5
while i > 0:
    print(i)
    i -= 1
```

① 1 2 3 4 5
② 5 4 3 2 1
③ 5 4 3 2 1 0
④ 무한 반복

정답	②
모범답안	② 5부터 1까지 감소하며 출력된다.

해설	i가 0이 되면 조건이 거짓이 되어 종료된다.

문항 22. 다음 중 range(10,0,-2)의 결과는?

① 10,8,6,4,2

② 10,8,6,4,2,0

③ 8,6,4,2,0

④ 10,9,8,7,6

정답	①
모범답안	① 10부터 2까지 2씩 감소한다.
해설	끝 값 0은 포함되지 않는다.

문항 23. 다음 코드의 실행 결과는?

```
count = 0
for i in range(4):
    count += 2
print(count)
```

① 2

② 4

③ 6

④ 8

정답	④
모범답안	④ 총 8이 된다.
해설	2를 네 번 더하면 8이다.

문항 24. 다음 코드의 출력 결과는?

```
for i in range(3):
    print('안녕')
```

① 1번

② 2번

③ 3번

④ 4번

정답	③

모범답안	③ '안녕'이 3번 출력된다.
해설	range(3)은 세 번 반복한다.

문항 25. 다음 코드의 출력 결과는?

```
for i in range(1,5):
    print(i * i)
```

① 1 2 3 4

② 1 4 9 16

③ 0 1 4 9

④ 1 8 27 64

정답	②
모범답안	② 제곱값 1,4,9,16이 출력된다.
해설	각 반복에서 i*i를 계산한다.

문항 26. 다음 코드의 출력 결과는?

```
i = 1
while i <= 5:
    if i == 4:
        break
    print(i)
    i += 1
```

① 1 2 3

② 1 2 3 4

③ 1 2 3 4 5

④ 무한 반복

정답	①
모범답안	① 1,2,3이 출력된다.
해설	i가 4가 되면 break로 종료된다.

문항 27. 다음 코드의 출력 결과는?

```
i = 0
```

```
while i < 5:
    i += 1
    if i == 3:
        continue
    print(i)
```

① 1 2 3 4 5

② 1 2 4 5

③ 0 1 2 4 5

④ 3 4 5

정답	②
모범답안	② 1,2,4,5가 출력된다.
해설	i가 3인 경우에만 print를 건너뛴다.

문항 28. 반복문과 함께 누적 계산에 자주 쓰이는 패턴은?

① sum = 0 후 반복마다 더하기

② if = 0 만들기

③ range = 0 만들기

④ print = 0 만들기

정답	①
모범답안	① 합계 계산의 대표적인 패턴이다.
해설	초기값을 정한 뒤 반복하면서 누적한다.

문항 29. 다음 코드의 출력 결과는?

```
for i in range(2,6):
    print(i, end=' ')
```

① 2 3 4 5

② 2 3 4 5 6

③ 1 2 3 4 5

④ 아무것도 출력 안 됨

정답	①
모범답안	① 2 3 4 5가 한 줄로 출력된다.
해설	end=' '는 줄바꿈 대신 공백을 붙인다.

문항 30. 다음 중 for문에 대한 설명으로 옳은 것은?

① 조건식만으로 반복한다.
② 정해진 범위나 시퀀스를 순회할 때 적합하다.
③ 반드시 무한 루프가 된다.
④ 문자열에는 사용할 수 없다.

정답	②
모범답안	② for문은 순회 기반 반복에 적합하다.
해설	range, 문자열, 리스트 등을 대상으로 사용할 수 있다.

문항 31. 다음 코드의 출력 결과는?

```
for i in range(3):
    print(i)
print('끝')
```

① 0 1 2
② 0 1 2 끝
③ 끝만
④ 오류

정답	②
모범답안	② 반복이 끝난 뒤 '끝'도 출력된다.
해설	반복문 밖의 문장은 반복 종료 후 한 번 실행된다.

문항 32. 다음 코드의 출력 결과는?

```
x = 0
while x < 3:
    x += 1
print(x)
```

① 0
② 1
③ 2
④ 3

정답	④
모범답안	④ 반복이 끝난 뒤 x 값은 3이다.
해설	0에서 시작해 1씩 증가하여 조건이 거짓이 될 때

	멈춘다.

문항 33. 다음 코드의 출력 결과는?

```
for i in range(3,0,-1):
    print(i)
```

① 1 2 3
② 3 2 1
③ 3 2 1 0
④ 0 1 2

정답	②
모범답안	② 3,2,1이 출력된다.
해설	감소하는 range를 사용한 예이다.

문항 34. 다음 중 횟수 반복의 예로 가장 적절한 것은?

① 사용자가 종료를 입력할 때까지 반복
② 비밀번호가 맞을 때까지 반복
③ 별표를 정확히 5번 출력
④ 조건이 참인 동안 반복

정답	③
모범답안	③ 정해진 횟수인 5번 출력은 횟수 반복의 전형적 예다.
해설	for문이 특히 적합하다.

문항 35. 다음 중 조건 반복의 예로 가장 적절한 것은?

① 1부터 10까지 출력
② 구구단 3단 출력
③ 온도가 목표값에 도달할 때까지 반복
④ 문자열 각 글자 출력

정답	③
모범답안	③ 조건이 만족될 때까지 반복하므로 while문에 가깝다.
해설	조건 반복은 반복 횟수가 미리 정해지지 않은 경우에 자주 쓰인다.

문항 36. 다음 코드의 출력 결과는?

```
for i in range(4):
    if i == 0:
        print('시작')
    print(i)
```

① 시작 0 1 2 3
② 0 1 2 3
③ 시작만
④ 오류

정답	①
모범답안	① '시작'이 먼저 출력되고 이어서 0~3이 출력된다.
해설	i가 0일 때만 조건문이 추가로 실행된다.

문항 37. 다음 코드의 출력 결과는?

```
for i in 'Hi':
    print(i * 2)
```

① HH ii
② HiHi
③ HH / ii
④ 오류

정답	③
모범답안	③ 각 문자를 두 번 반복한 HH와 ii가 줄바꿈되어 출력된다.
해설	문자열을 순회하면서 문자 연산을 수행했다.

문항 38. 다음 코드의 출력 결과는?

```
for i in range(1,4):
    for j in range(1,3):
        print(i, j)
```

① 2개만 출력
② 6개 쌍이 출력
③ 3개만 출력

④ 오류

정답	②
모범답안	② 3 x 2 = 6개의 쌍이 출력된다.
해설	바깥 반복 3번, 안쪽 반복 2번이 결합된다.

문항 39. 다음 코드의 출력 결과는?

```
s = 1
for i in range(1,4):
    s *= i
print(s)
```

① 3

② 4

③ 6

④ 9

정답	③
모범답안	③ 1 x 2 x 3 = 6이다.
해설	곱셈 누적 계산 예시이다.

문항 40. 다음 중 break 사용 목적에 가장 가까운 것은?

① 현재 반복문 전체를 중단

② 현재 반복만 건너뜀

③ 함수 종료

④ 조건식을 만듦

정답	①
모범답안	① break는 반복문 전체를 끝낸다.
해설	continue와 혼동하지 않는 것이 중요하다.

문항 41. 다음 중 continue 사용 목적에 가장 가까운 것은?

① 반복문 전체 종료

② 현재 반복을 건너뛰고 다음 반복 진행

③ 프로그램 종료

④ 변수 초기화

정답	②
모범답안	② continue는 특정 조건의 현재 회차만 생략한다.

해설	필터링처럼 일부 경우를 건너뛸 때 유용하다.

문항 42. 다음 코드의 출력 결과는?

```
i = 0
while i < 4:
    print(i)
    i += 2
```

① 0 1 2 3
② 0 2
③ 2 4
④ 0 2 4

정답	②
모범답안	② 0과 2가 출력된다.
해설	i가 0,2일 때만 조건을 만족한다.

문항 43. 다음 코드의 출력 결과는?

```
for i in range(5):
    if i % 2 == 1:
        print(i)
```

① 0 2 4
② 1 3
③ 1 2 3 4
④ 5

정답	②
모범답안	② 홀수인 1과 3만 출력된다.
해설	나머지가 1인 경우가 홀수이다.

문항 44. 다음 중 range(4,9)의 결과는?

① 4,5,6,7,8
② 4,5,6,7,8,9
③ 5,6,7,8,9
④ 4,6,8

정답	①
모범답안	① 4부터 8까지이다.
해설	끝 값 9는 포함되지 않는다.

문항 45. 다음 코드의 출력 결과는?

```
for i in range(2):
    print('A')
    print('B')
```

① A B 한 번
② A A B B
③ A B / A B
④ 오류

정답	③
모범답안	③ 각 반복마다 A, B가 차례로 출력되어 두 번 반복된다.
해설	블록에 있는 두 문장이 반복마다 함께 실행된다.

문항 46. 다음 코드의 출력 결과는?

```
for i in range(1):
    print('X')
```

① X
② XX
③ 출력 없음
④ 오류

정답	①
모범답안	① X가 한 번 출력된다.
해설	range(1)은 0 하나만 생성하므로 한 번 반복한다.

문항 47. 다음 중 반복문 작성 시 가장 중요한 것 중 하나는?

① 항상 변수 이름을 x로 짓는 것
② 종료 조건이나 반복 범위를 분명히 하는 것
③ 모든 반복문에 break를 넣는 것
④ 반드시 중첩 반복문을 사용하는 것

정답	②
모범답안	② 반복 횟수나 종료 조건이 분명해야 올바르게 동작한다.
해설	특히 while문은 종료 조건을 놓치면 무한 루프가 생길 수 있다.

문항 48. 다음 코드의 출력 결과는?

```
text = 'abc'
for ch in text:
    print(ch.upper())
```

① A B C
② abc
③ ABC
④ 오류

정답	①
모범답안	① A, B, C가 차례로 출력된다.
해설	각 문자를 대문자로 바꿔 출력했다.

문항 49. 다음 코드의 출력 결과는?

```
n = 3
while n:
    print(n)
    n -= 1
```

① 3 2 1
② 3 2 1 0
③ 0 1 2 3
④ 무한 반복

정답	①
모범답안	① 3, 2, 1이 출력된다.
해설	n이 0이 되면 False로 평가되어 반복이 끝난다.

문항 50. 다음 코드의 출력 결과는?

```
for i in range(3):
    for j in range(1):
        print('*')
```

① 별 1개
② 별 2개
③ 별 3개
④ 별 4개

정답	③
모범답안	③ 바깥 반복 3회 x 안쪽 반복 1회로 별이 3개 출력된다.
해설	중첩 반복문이지만 안쪽 반복 횟수가 1이다.

함수

제5장 핵심 개념 기반 80문항

문항 1. 다음 중 함수의 가장 중요한 역할은?

① 변수 값을 자동 증가
② 반복되는 코드를 묶어 재사용
③ 오류를 자동 수정
④ 반드시 프로그램 길이 감소

정답	②
모범답안	② 반복되는 코드를 묶어 재사용한다.
해설	함수는 프로그램을 구조화하고 같은 작업을 여러 번 활용하게 해 준다.

문항 2. 파이썬에서 함수를 정의할 때 사용하는 키워드는?

① function
② define
③ def
④ func

정답	③
모범답안	③ def를 사용한다.
해설	사용자 정의 함수는 def 함수이름(): 형식으로 만든다.

문항 3. 다음 중 올바른 함수 정의 형식은?

① def hello()
② def hello:
③ def hello():
④ function hello():

정답	③
모범답안	③ def hello(): 가 올바른 형식이다.
해설	함수 머리글 끝에는 콜론이 필요하다.

문항 4. 함수 hello를 호출하는 방법으로 옳은 것은?

① call hello
② hello[]
③ hello()

④ def hello()

정답	③
모범답안	③ 함수 이름 뒤에 괄호를 붙여 호출한다.
해설	괄호는 함수 실행을 의미한다.

문항 5. 함수에 전달되는 실제 값을 무엇이라고 하는가?

① 매개변수
② 인수
③ 반환값
④ 블록

정답	②
모범답안	② 호출 시 전달하는 실제 값을 인수라고 한다.
해설	정의부의 변수 이름은 매개변수이다.

문항 6. 함수 정의에서 값을 받는 변수는 무엇인가?

① 반환값
② 매개변수
③ 출력문
④ 예약어

정답	②
모범답안	② 매개변수는 함수 정의에서 입력값을 받는 변수이다.
해설	인수와 매개변수의 구분은 함수 단원에서 매우 중요하다.

문항 7. 함수 결과를 돌려줄 때 사용하는 키워드는?

① print
② input
③ return
④ break

정답	③
모범답안	③ return을 사용한다.
해설	return은 값을 돌려주고 함수 실행을 종료한다.

문항 8. 다음 중 내장함수의 예는?

① print()
② myfunc()
③ hello()
④ test()

정답	①
모범답안	① print()는 파이썬이 기본 제공하는 내장함수이다.
해설	사용자 정의 함수는 직접 def로 만들어야 한다.

문항 9. 함수를 사용하는 가장 큰 이유로 적절한 것은?

① 들여쓰기 제거
② 코드 중복 감소
③ 변수 금지
④ 연산자 제거

정답	②
모범답안	② 같은 코드를 여러 번 쓰지 않게 해 준다.
해설	유지보수성과 가독성도 함께 좋아진다.

문항 10. 파이썬에서 함수 본문을 구분하는 기준은?

① 세미콜론
② 중괄호
③ 들여쓰기
④ 괄호

정답	③
모범답안	③ 들여쓰기로 함수 블록을 구분한다.
해설	조건문, 반복문과 마찬가지로 함수도 들여쓰기가 중요하다.

문항 11. 다음 코드의 실행 결과는?

```
def hello():
    print('안녕')

hello()
```

① hello
② 안녕

③ 출력 없음

④ 오류

정답	②
모범답안	② 안녕이 출력된다.
해설	함수를 호출하면 함수 내부의 print 문장이 실행된다.

문항 12. 다음 코드의 실행 결과는?

```
def add(a, b):
    print(a + b)

add(2, 3)
```

① 2

② 3

③ 5

④ 오류

정답	③
모범답안	③ 5가 출력된다.
해설	2와 3이 전달되어 합이 출력된다.

문항 13. 다음 함수의 매개변수 개수는?

```
def area(width, height):
    return width * height
```

① 0개

② 1개

③ 2개

④ 3개

정답	③
모범답안	③ width와 height 두 개의 매개변수가 있다.
해설	함수 정의 괄호 안에 있는 이름의 수를 세면 된다.

문항 14. 다음 중 함수 이름으로 사용할 수 있는 것은?

① 3sum

② my-func

③ class

④ my_func

정답	④
모범답안	④ my_func는 올바른 식별자이다.
해설	숫자로 시작하거나 하이픈을 쓰거나 예약어를 쓰면 안 된다.

문항 15. 인수와 매개변수에 대한 설명으로 옳은 것은?

① 인수는 함수 안 이름이다.

② 매개변수는 실제 값이다.

③ 인수는 호출 때 주는 값, 매개변수는 정의에서 받는 변수이다.

④ 둘은 같은 개념이 아니다.

정답	③
모범답안	③ 이 설명이 정확하다.
해설	예: add(2,3)에서 2,3은 인수이고 def add(a,b)의 a,b는 매개변수다.

문항 16. 다음 코드의 실행 결과는?

```
def double(x):
    return x * 2

print(double(3))
```

① 3

② 5

③ 6

④ 9

정답	③
모범답안	③ 6이 출력된다.
해설	double(3)은 3*2 결과인 6을 반환한다.

문항 17. 다음 코드의 실행 결과는?

```
def f():
    return 10
```

```
x = f()
print(x)
```

① 0
② 1
③ 10
④ 오류

정답	③
모범답안	③ x에 10이 저장되어 출력된다.
해설	함수 반환값을 변수에 저장해 다시 사용할 수 있다.

문항 18. 다음 코드의 실행 결과는?

```
def star():
    print('*')

star()
star()
```

① 별 1번
② 별 2번
③ ** 한 줄
④ 출력 없음

정답	②
모범답안	② star()를 두 번 호출했으므로 별이 두 번 출력된다.
해설	함수는 정의 후 여러 번 호출할 수 있다.

문항 19. return문이 없는 함수가 끝나면 기본적으로 반환되는 값은?

① 0
② False
③ None
④ 빈 문자열

정답	③
모범답안	③ None이 기본 반환값이다.
해설	명시적인 return이 없더라도 함수는 None을 돌려준다.

문항 20. 다음 코드의 실행 결과는?

```
print(len('abc'))
```

① 2
② 3
③ 4
④ 오류

정답	②
모범답안	② 문자열 길이 3이 출력된다.
해설	len()은 길이를 구하는 대표적 내장함수이다.

문항 21. 사용자 정의 함수에 대한 설명으로 옳은 것은?

① 파이썬이 미리 제공한다.
② 사용자가 직접 만든 함수이다.
③ def 없이 자동 생성된다.
④ 내장함수와 함께 못 쓴다.

정답	②
모범답안	② def로 직접 작성한 함수가 사용자 정의 함수이다.
해설	필요한 기능을 사용자가 설계해 만들 수 있다.

문항 22. 다음 코드의 실행 결과는?

```
def sub(a, b):
    return a - b

print(sub(7, 2))
```

① 9
② 5
③ -5
④ 오류

정답	②
모범답안	② 7 - 2 = 5 이다.
해설	return은 계산 결과를 호출한 곳으로 돌려준다.

문항 23. 지역변수에 대한 설명으로 옳은 것은?

① 프로그램 전체에서 사용
② 함수 밖에서만 생성
③ 함수 안에서 만들어져 그 함수 안에서 사용
④ 반드시 global 필요

정답	③
모범답안	③ 지역변수는 함수 내부에서 사용되는 변수이다.
해설	함수 밖에서는 직접 접근할 수 없다.

문항 24. 다음 코드의 출력 결과는?

```
gx = 100

def myfunc():
    gx = 200
    print(gx)

myfunc()
print(gx)
```

① 100,100
② 200,100
③ 100,200
④ 200,200

정답	②
모범답안	② 함수 안의 gx는 지역변수라 200, 전역 gx는 그대로 100이다.
해설	같은 이름이라도 함수 안에서 새로 대입하면 지역변수로 취급된다.

문항 25. 다음 코드의 출력 결과는?

```
gx = 100

def myfunc():
```

```
    global gx
    gx = 200
    print(gx)

myfunc()
print(gx)
```

① 100,100
② 200,100
③ 100,200
④ 200,200

정답	④
모범답안	④ global gx를 사용했으므로 전역변수 자체가 200으로 바뀐다.
해설	함수 안과 밖 모두 같은 전역 gx를 가리킨다.

문항 26. 함수 안에서 변수에 값을 대입하면 특별한 선언이 없는 한 보통 무엇이 되는가?

① 상수
② 지역변수
③ 전역변수
④ 예약어

정답	②
모범답안	② 기본적으로 지역변수로 취급된다.
해설	전역변수를 수정하려면 global 선언이 필요하다.

문항 27. 다음 중 함수에 대한 설명으로 옳은 것은?

① 다른 함수를 호출할 수 없다.
② 한 번만 호출 가능
③ 다른 함수를 호출할 수 있다.
④ 반복문 안에서 사용할 수 없다.

정답	③
모범답안	③ 함수는 다른 함수를 자유롭게 호출할 수 있다.
해설	큰 문제를 여러 함수로 나누어 해결하는 데 유리하다.

문항 28. 재귀호출에 대한 설명으로 가장 옳은 것은?

① 함수가 자신을 다시 호출
② 함수가 자동 두 번 실행
③ 함수 여러 개 동시 생성
④ 함수 안 반복문 사용

정답	①
모범답안	① 자기 자신을 다시 부르는 것이 재귀호출이다.
해설	같은 구조의 작은 문제로 나눌 때 유용하다.

문항 29. 다음 코드의 실행 결과는?

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)

print(fact(3))
```

① 3
② 5
③ 6
④ 9

정답	③
모범답안	③ 3! = 6 이다.
해설	fact(3)=3*fact(2)=3*2*fact(1)=6이다.

문항 30. 재귀함수에서 반드시 필요한 요소는?

① 무한 반복
② 종료 조건
③ 전역변수
④ 입력문

정답	②
모범답안	② 종료 조건이 있어야 재귀가 멈춘다.
해설	없으면 계속 자기 자신을 호출하게 된다.

문항 31. 다음 코드의 실행 결과는?

```
def square(x):
    return x * x

print(square(4) + 1)
```

① 8
② 15
③ 16
④ 17

정답	④
모범답안	④ 16 + 1 = 17 이다.
해설	반환값은 다른 계산식에 바로 사용할 수 있다.

문항 32. 다음 코드의 실행 결과는?

```
def hello(name):
    print('안녕', name)

hello('철수')
```

① 안녕
② 철수
③ 안녕 철수
④ 오류

정답	③
모범답안	③ '안녕 철수'가 출력된다.
해설	매개변수 name에 '철수'가 전달된다.

문항 33. 다음 코드에서 오류가 발생하는 호출은?

```
def show(a, b):
    print(a, b)
```

① show(1, 2)
② show('A', 'B')
③ show(3, 4)
④ show(1)

정답	④

모범답안	④ 매개변수는 2개인데 인수는 1개라서 오류가 난다.
해설	호출 시 인수 개수는 매개변수 개수와 맞아야 한다.

문항 34. 다음 코드의 실행 결과는?

```
def add3(a, b, c):
    return a + b + c

print(add3(1, 2, 3))
```

① 3
② 5
③ 6
④ 7

정답	③
모범답안	③ 세 수의 합은 6이다.
해설	매개변수 세 개를 받아 합을 반환한다.

문항 35. 다음 코드의 실행 결과는?

```
def f(x):
    x = x + 1

a = 10
f(a)
print(a)
```

① 9
② 10
③ 11
④ 오류

정답	②
모범답안	② a 자체는 바뀌지 않는다.
해설	함수 안에서 바뀐 것은 지역변수 x이다.

문항 36. 매개변수 이름에 대한 설명으로 옳은 것은?

① 함수 밖 변수명과 반드시 같아야 한다.
② 함수 밖 변수명과 달라도 된다.
③ 숫자만 가능하다.
④ 항상 x, y만 써야 한다.

정답	②
모범답안	② 매개변수 이름은 함수 내부에서 쓰이는 이름이므로 바깥 이름과 달라도 된다.
해설	의미가 드러나는 이름을 쓰는 것이 좋다.

문항 37. 다음 코드의 실행 결과는?

```
print(max(3, 7, 5))
```

① 3
② 5
③ 7
④ 15

정답	③
모범답안	③ max는 가장 큰 값을 반환한다.
해설	세 값 중 최댓값은 7이다.

문항 38. 다음 코드의 실행 결과는?

```
print(abs(-8))
```

① -8
② 0
③ 8
④ 오류

정답	③
모범답안	③ abs는 절댓값을 반환한다.
해설	-8의 절댓값은 8이다.

문항 39. 다음 코드의 실행 결과는?

```
print(min(4, 9, 1))
```

① 1

② 4

③ 9

④ 14

정답	①
모범답안	① 최솟값 1이 출력된다.
해설	min은 가장 작은 값을 찾는 내장함수이다.

문항 40. 다음 코드의 실행 결과는?

```
print(sum([1, 2, 3]))
```

① 3

② 5

③ 6

④ 오류

정답	③
모범답안	③ 리스트 원소 합 6이 출력된다.
해설	sum 함수는 수치형 원소의 합계를 계산한다.

문항 41. 재귀호출을 잘못 작성했을 때 발생하기 쉬운 문제는?

① 자동 형변환

② 무한 재귀

③ 들여쓰기 제거

④ 이름 변경

정답	②
모범답안	② 종료 조건이 없으면 무한 재귀가 발생할 수 있다.
해설	결국 호출 한도를 넘어서 오류가 발생한다.

문항 42. 종료 조건이 없는 재귀함수에 대한 설명으로 옳은 것은?

① 한 번만 실행

② 항상 0 반환

③ 계속 자기 자신을 호출하다 문제가 생길 수 있다.

④ 자동으로 반복문으로 바뀐다.

정답	③
모범답안	③ 재귀는 멈추는 조건이 없으면 끝나지 않는다.

해설	반드시 base case를 마련해야 한다.

문항 43. 다음 코드의 실행 결과는?

```
def total(n):
    if n == 0:
        return 0
    return n + total(n - 1)

print(total(3))
```

① 3
② 5
③ 6
④ 9

정답	③
모범답안	③ 3+2+1+0 = 6이다.
해설	재귀로 1부터 n까지의 합을 구한 예시이다.

문항 44. 함수 헤더(header)에 포함되는 요소로 가장 적절한 것은?

① 함수 이름과 매개변수
② 반복문과 조건문
③ 주석과 문자열
④ 리스트와 튜플

정답	①
모범답안	① def 함수이름(매개변수): 부분이 헤더이다.
해설	실제 실행 문장은 들여쓰기된 본문에 들어간다.

문항 45. 다음 중 매개변수 1개를 가지는 올바른 함수 정의는?

① def show(x):
② def show x:
③ show(def x):
④ function show(x):

정답	①
모범답안	① 이 형식이 올바르다.
해설	파이썬은 def 키워드와 괄호, 콜론을 사용한다.

문항 46. return문에 대한 설명으로 가장 옳은 것은?

① 함수 안에서 쓸 수 없다.
② 값을 반환하고 함수 실행을 끝낸다.
③ 반드시 두 번 써야 한다.
④ print와 완전히 같다.

정답	②
모범답안	② return은 반환과 종료를 동시에 수행한다.
해설	print는 단순 출력이고 return과 역할이 다르다.

문항 47. 다음 코드의 실행 결과는?

```
def f():
    return 1
    print(2)

print(f())
```

① 1
② 2
③ 1 2
④ 오류

정답	①
모범답안	① 1만 출력된다.
해설	return 뒤의 문장은 실행되지 않는다.

문항 48. 다음 코드의 실행 결과는?

```
def even(n):
    return n % 2 == 0

print(even(4))
```

① 4
② 0
③ True

④ False

정답	③
모범답안	③ 4는 짝수이므로 True가 반환된다.
해설	비교식 결과 자체를 return할 수 있다.

문항 49. 지역변수와 전역변수에 대한 설명으로 옳은 것은?

① 지역변수는 어디서나 사용 가능

② 전역변수는 함수 안에서 절대 읽지 못함

③ 지역변수는 함수 내부에서 생성되어 사용됨

④ 전역변수는 반드시 함수 안에서만 선언

정답	③
모범답안	③ 지역변수는 함수 안에서만 유효하다.
해설	전역변수는 함수 밖에서 만든 변수로, 함수 안에서 읽는 것은 가능하다.

문항 50. 다음 코드의 실행 결과는?

```
def add(a, b):
    return a + b

result = add(2, 3)
print(result * 2)
```

① 5

② 6

③ 10

④ 12

정답	③
모범답안	③ add 결과 5에 2를 곱해 10이 된다.
해설	반환값은 변수에 저장해서 재사용할 수 있다.

문항 51. 다음 코드의 실행 결과는?

```
print(pow(2, 3))
```

① 5

② 6

③ 8

④ 9

정답	③
모범답안	③ 2의 3제곱은 8이다.
해설	pow는 거듭제곱 계산 내장함수이다.

문항 52. 다음 코드의 실행 결과는?

```
print(len([10, 20, 30]))
```

① 2

② 3

③ 10

④ 30

정답	②
모범답안	② 리스트 원소 개수 3이 출력된다.
해설	len은 문자열뿐 아니라 리스트 길이도 구할 수 있다.

문항 53. 매개변수의 역할로 가장 적절한 것은?

① 반복 횟수만 정함

② 함수 바깥의 값을 함수 안으로 전달받음

③ 함수 이름을 바꿈

④ 반환값을 자동 출력

정답	②
모범답안	② 매개변수는 외부 입력값을 받는 통로이다.
해설	같은 함수라도 다른 값을 넣어 다르게 활용할 수 있다.

문항 54. 함수가 반환한 값을 활용하는 방법으로 옳은 것은?

① 변수에 저장해 다시 사용

② 항상 버려짐

③ 출력문에서만 사용

④ 문자열만 가능

정답	①
모범답안	① 반환값은 여러 방식으로 재활용 가능하다.
해설	계산, 비교, 다른 함수 인수로도 사용할 수 있다.

문항 55. 함수 안에서 전역변수 값을 변경하려고 할 때 쓰는 키워드는?

① local
② return
③ global
④ def

정답	③
모범답안	③ global 키워드로 전역변수를 수정하겠다고 선언한다.
해설	읽기만 할 때는 필요 없는 경우가 많지만, 대입할 때는 중요하다.

문항 56. 다음 코드의 실행 결과는?

```
def myfunc():
    x = 5
    print(x)

myfunc()
```

① 5
② x
③ 출력 없음
④ 오류

정답	①
모범답안	① 5가 출력된다.
해설	지역변수 x를 함수 안에서 출력한 예이다.

문항 57. 다음 코드의 실행 결과는?

```
def hello():
    print('A')
    return
    print('B')
```

hello()

① A

② B

③ A B

④ 출력 없음

정답	①
모범답안	① A만 출력된다.
해설	return을 만나는 순간 함수가 종료된다.

문항 58. 재귀호출이 특히 적합한 경우는?

① 같은 형태의 더 작은 문제로 나눌 수 있을 때

② 출력을 쓰지 않을 때

③ 전역변수가 많을 때

④ 반복문을 쓸 수 없을 때만

정답	①
모범답안	① 재귀는 문제를 작은 동일 구조로 나눌 때 효과적이다.
해설	대표적으로 팩토리얼, 트리 탐색 등이 있다.

문항 59. 다음 중 내장함수가 아닌 것은?

① len()

② abs()

③ print()

④ mysum()

정답	④
모범답안	④ mysum()은 사용자가 따로 만들지 않으면 내장함수가 아니다.
해설	나머지 셋은 파이썬이 기본 제공한다.

문항 60. 함수 사용의 장점으로 가장 적절한 것은?

① 프로그램 구조화와 가독성 향상

② 모든 변수를 전역으로 변경

③ 오류 절대 방지

④ 반복문 사용 금지

정답	①
모범답안	① 기능별 분리와 재사용성 향상이 함수의 핵심 장점이다.
해설	긴 프로그램을 관리하기 쉬워진다.

문항 61. 다음 코드의 실행 결과는?

```
def greet():
    return 'hi'

print(greet())
```

① greet
② hi
③ None
④ 오류

정답	②
모범답안	② hi가 출력된다.
해설	함수 greet가 문자열 'hi'를 반환했다.

문항 62. 다음 코드의 실행 결과는?

```
def mul(a, b):
    return a * b

print(mul(4, 5))
```

① 9
② 20
③ 45
④ 오류

정답	②
모범답안	② 4 * 5 = 20 이다.
해설	곱셈 결과를 반환한다.

문항 63. 다음 코드의 실행 결과는?

```
def check(x):
    if x > 0:
        return '양수'
    else:
        return '0 또는 음수'

print(check(-2))
```

① 양수
② 0 또는 음수
③ -2
④ 오류

정답	②
모범답안	② -2는 양수가 아니므로 else 쪽 문자열을 반환한다.
해설	조건에 따라 서로 다른 값을 반환하는 함수이다.

문항 64. 다음 코드의 실행 결과는?

```
def first():
    print('A')

def second():
    first()
    print('B')

second()
```

① A만
② B만
③ A 다음 B
④ 오류

정답	③
모범답안	③ second가 first를 호출하므로 A가 먼저, 그 다음

	B가 출력된다.
해설	함수 안에서 다른 함수를 호출할 수 있음을 보여준다.

문항 65. 다음 코드의 실행 결과는?

```
def f(x, y):
    return x > y

print(f(3, 1))
```

① 3
② 1
③ True
④ False

정답	③
모범답안	③ 3 > 1은 참이므로 True가 반환된다.
해설	함수는 불리언 값도 반환할 수 있다.

문항 66. 다음 코드의 실행 결과는?

```
def no_return():
    x = 3

print(no_return())
```

① 3
② 0
③ None
④ 오류

정답	③
모범답안	③ 명시적 return이 없으므로 None이 출력된다.
해설	함수 내부 계산만 하고 결과를 반환하지 않은 경우이다.

문항 67. 다음 코드의 실행 결과는?

```
def inc(n):
    return n + 1

print(inc(inc(3)))
```

① 4
② 5
③ 6
④ 7

정답	②
모범답안	② 안쪽 inc(3)=4, 바깥 inc(4)=5 이다.
해설	반환값을 다시 함수 인수로 넣을 수 있다.

문항 68. 다음 코드의 실행 결과는?

```
def f(a):
    print(a)

x = 7
f(x)
```

① a
② 7
③ x
④ 오류

정답	②
모범답안	② x 값 7이 매개변수 a로 전달되어 출력된다.
해설	변수 자체가 아니라 그 값이 인수로 전달된다.

문항 69. 다음 코드의 실행 결과는?

```
def same(x):
    return x
```

print(same('파이썬'))

① same
② 파이썬
③ None
④ 오류

정답	②
모범답안	② 전달된 문자열이 그대로 반환된다.
해설	입력값을 그대로 돌려주는 단순 함수이다.

문항 70. 다음 코드의 실행 결과는?

```
def compare(a, b):
    if a == b:
        return '같음'
    return '다름'

print(compare(2, 3))
```

① 같음
② 다름
③ 2
④ 3

정답	②
모범답안	② 두 값이 다르므로 '다름'이 반환된다.
해설	조건 함수의 기본 예시이다.

문항 71. 다음 코드의 실행 결과는?

```
def countdown(n):
    if n == 0:
        return 0
    print(n)
    return countdown(n - 1)
```

countdown(2)

① 2와 1 출력
② 2만 출력
③ 1만 출력
④ 출력 없음

정답	①
모범답안	① 2와 1이 차례로 출력된다.
해설	재귀 호출 전에 print를 수행했기 때문이다.

문항 72. 다음 중 함수 설계 시 좋은 습관으로 가장 적절한 것은?

① 의미 없는 이름 사용
② 한 함수에 너무 많은 일을 몰아넣기
③ 역할이 드러나는 이름과 한 가지 책임 중심 설계
④ 반드시 전역변수 사용

정답	③
모범답안	③ 함수는 역할이 분명할수록 재사용과 유지보수가 쉽다.
해설	좋은 이름은 가독성을 크게 높여 준다.

문항 73. 다음 코드의 실행 결과는?

```
x = 10

def show():
    print(x)

show()
```

① 10
② x
③ 오류
④ 출력 없음

정답	①
모범답안	① 함수 안에서 전역변수 x를 읽어 10을 출력한다.
해설	전역변수를 읽기만 하는 경우에는 global 선언이

	보통 필요 없다.

문항 74. 다음 코드의 실행 결과는?

```
def outer():
    x = 5
    print(x)

outer()
print(x)
```

① 5와 5
② 5와 오류
③ 오류와 5
④ 아무것도 출력 안 됨

정답	②
모범답안	② 함수 안에서는 5가 출력되지만, 함수 밖 print(x)는 오류가 난다.
해설	지역변수 x는 함수 밖에서 접근할 수 없다.

문항 75. 다음 코드의 실행 결과는?

```
def add(a, b):
    c = a + b
    return c

print(add(1, 4))
```

① 4
② 5
③ 14
④ 오류

정답	②
모범답안	② 지역변수 c에 계산 결과 5를 저장한 뒤 반환한다.
해설	중간 결과를 지역변수에 담아 반환할 수 있다.

문항 76. 다음 코드의 실행 결과는?

```
def zero():
    return 0

if zero():
    print('참')
else:
    print('거짓')
```

① 참
② 거짓
③ 0
④ 오류

정답	②
모범답안	② 숫자 0은 False로 평가되어 '거짓'이 출력된다.
해설	함수 반환값도 조건식에 바로 사용할 수 있다.

문항 77. 다음 코드의 실행 결과는?

```
def one():
    return 1

if one():
    print('참')
else:
    print('거짓')
```

① 참
② 거짓
③ 1
④ 오류

정답	①
모범답안	① 숫자 1은 True로 평가된다.
해설	반환값을 조건문과 결합한 예시이다.

문항 78. 다음 코드의 실행 결과는?

```
def message():
    print('hello')

result = message()
print(result)
```

① hello만
② hello 다음 None
③ None만
④ 오류

정답	②
모범답안	② 함수가 hello를 출력한 뒤, 반환값이 없으므로 result에는 None이 저장된다.
해설	print와 return의 차이를 구분해야 한다.

문항 79. 다음 중 함수 호출에 대한 설명으로 옳은 것은?

① 정의한 뒤 호출해야 한다.
② 호출만 하면 자동 정의된다.
③ 괄호 없이 호출해야 한다.
④ 인수는 항상 필요하다.

정답	①
모범답안	① 함수는 먼저 정의된 후 호출되어야 한다.
해설	매개변수가 없는 함수는 인수 없이 호출할 수 있다.

문항 80. 다음 코드의 실행 결과는?

```
def add(a, b):
    print(a + b)

result = add(1, 2)
print(result)
```

① 3만
② 3 다음 None
③ None만

④ 오류

정답	②
모범답안	② add는 합계 3을 출력하지만 반환하지 않으므로 result는 None이다.
해설	print와 return을 혼동하지 않도록 주의해야 한다.